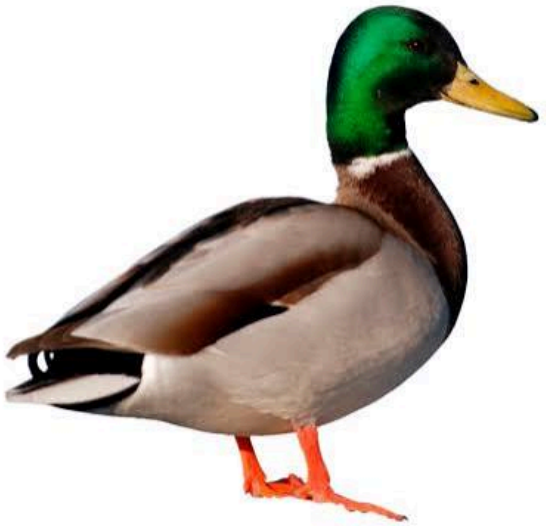
Ciência de Dados Aplicada ao Direito I

Aula 8

Condicionais: dois artigos viram condição



Prof. Bruno da Cunha de Oliveira - 2026/2

O que vocês estão achando do curso

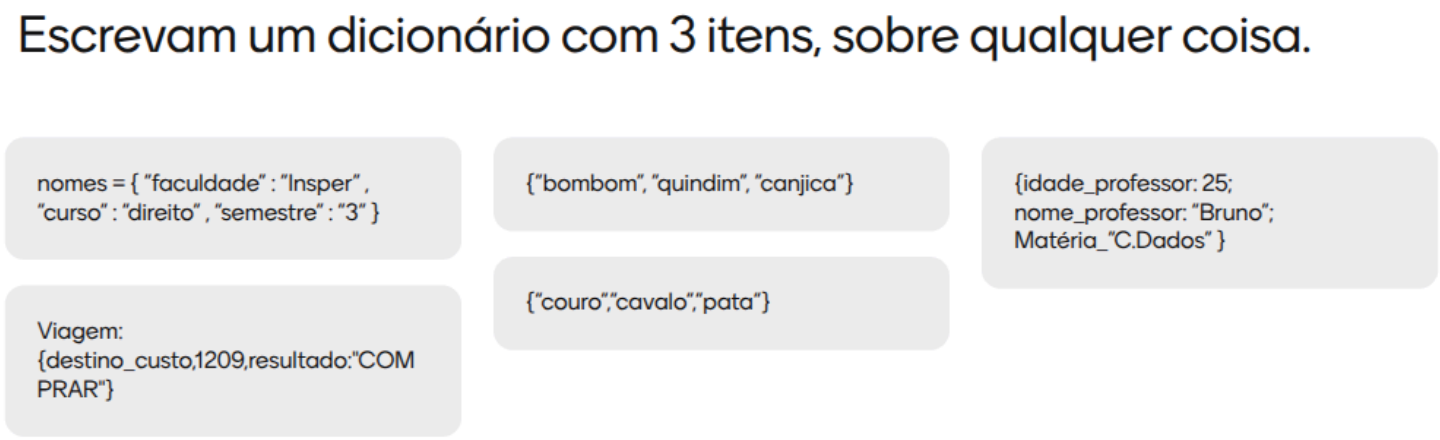
Três semanas e sete aulas. Antes de seguir, eu quero ouvir:

1. o que está funcionando;
2. o que está atrapalhando;
3. o que vocês mudariam, e como.

Sugestão de melhoria que mude alguma coisa no curso vale 0,1. Sem limite por aluno e fora do teto de 1,0 de bonificação: vale na avaliação intermediária ou na final.

Correção de ontem: a chave do dicionário é texto

Ontem eu disse que a chave não era texto. Errei, e a culpa é do pseudocódigo: ele não tem gabarito fixo, e a grafia que eu usava até então escrevia a chave sem aspas. Em Python, as respostas com aspas estavam certas.



Insper

A mesma palavra, dois papéis

```
idade_autor = 67
```

À esquerda do `=`, sem aspas: é o nome de uma variável.

```
{"idade_autor": 67}
```

À esquerda do `:`, com aspas: é uma chave, e chave é texto.

Em Python, `{idade_autor: 67}` não usa o texto `idade_autor` como chave: usa o valor da variável com esse nome, se ela existir, e dá erro se não existir.

O caput do art. 171

Insper

Uma conjunção que decide a pena

Oitava Emenda (EUA, 1791)		Constituição da Califórnia, art. 1º, §17
"nor cruel **and** unusual punishments inflicted"	VS	"Cruel **or** unusual punishment may not be inflicted"
A pena precisa ser cruel e incomum para ser vedada. Basta não ser incomum, e ela passa.		Basta ser uma das duas para ser vedada.

Quase as mesmas palavras. Uma conjunção diferente, e a pena que passa de um lado não passa do outro.

Sem parênteses, a frase tem duas leituras

Uma regra de fila preferencial, lida em voz alta: “tem mais de 60 anos e é aposentado ou pensionista”. Escrita sem parênteses:

```
tem_mais_de_60 E aposentado OU pensionista
```

Chega uma pensionista de 45 anos. Ela entra na fila?

Se o `OU` vale só para “aposentado ou pensionista”, ela precisa ter mais de 60: não entra.

Se o `OU` vale para a frase inteira, ser pensionista já basta: entra.

A mesma linha, dois resultados. O computador não escolhe a leitura: ou ele segue uma regra fixa que você não escreveu, ou dá erro.

Os parênteses dizem onde o OU acaba

A mesma regra, agora com os parênteses dizendo qual leitura vale:

```
(tem_mais_de_60 E aposentado)
  OU pensionista
```

Pensionista de 45 anos: VERDADEIRO.

```
tem_mais_de_60
  E (aposentado OU pensionista)
```

Pensionista de 45 anos: FALSO.

O parêntese diz a quantas coisas cada `OU` se aplica. Quem escreve escolhe a leitura, e o computador obedece.

Regra prática: todo `OU` que está dentro de uma exigência vai entre parênteses.

Quatro exigências num período só

Código Penal. Art. 171, caput e pena.

> **Art. 171 -** Obter, para si ou para outrem, vantagem ilícita, em prejuízo alheio, induzindo ou mantendo alguém em erro, mediante artifício, ardil, ou qualquer outro meio fraudulento:
>
> Pena - reclusão, de 1 (um) a 5 (cinco) anos, e multa.

O caput é a condição. A pena é o que vem depois do `ENTÃO`.

O caput vira condição

Em grupos, reescrevam o caput do art. 171 como condição:

1. separem o que precisa acontecer junto do que admite uma opção entre várias
2. escrevam `SE ... ENTÃO ...`, com `E`, `OU` e os parênteses no lugar certo, e a pena depois do `ENTÃO`.

Entreguem pelo formulário “art. 171” no Blackboard, com o nome completo de todos os integrantes. Dez minutos.

A pista está nas vírgulas e nos “ou”.

O caput vira condição

RESPOSTA

obter vantagem ilícita

E

em prejuízo alheio

E

induzir **ou** manter alguém em erro

E

artifício **ou** ardil **ou** outro meio fraudulento

As quatro caixas somam: falta uma, não há estelionato. Dentro das duas últimas, basta uma alternativa.

A condição escrita

RESPOSTA

```
SE   obteve_vantagem_ilicita
 E   houve_prejuizo_alheio
 E   (induziu_em_erro OU manteve_em_erro)
 E   (usou_artificio OU usou_ardil
      OU usou_outro_meio_fraudulento)
ENTÃO
    reclusão de 1 a 5 anos, e multa
```

Os nomes podem variar. Os parênteses, não: eles dizem a quantas coisas cada `OU` se aplica.

Um E virando OU

E se o prejuízo alheio deixasse de somar e passasse a ser mais uma alternativa? Só a segunda linha muda.

Como está

```
SE   obteve_vantagem_ilicita
 E   houve_prejuizo_alheio
 E   (induziu_em_erro
      OU manteve_em_erro)
 E   (usou_artificio OU usou_ardil
      OU outro_meio_fraudulento)
ENTÃO reclusão de 1 a 5 anos, e multa
```

Com um E trocado por OU

```
SE   obteve_vantagem_ilicita
 E   (houve_prejuizo_alheio
      OU induziu_em_erro
      OU manteve_em_erro)
 E   (usou_artificio OU usou_ardil
      OU outro_meio_fraudulento)
ENTÃO reclusão de 1 a 5 anos, e multa
```

À direita, quem usa um ardil, induz alguém em erro e obtém vantagem sem que ninguém perca nada já responde. À esquerda, não: o prejuízo alheio é exigência, e não opção.

Um caractere de diferença, e a fronteira do tipo penal se move.

O NÃO é o lado da defesa

A acusação precisa de todas as caixas. A defesa precisa derrubar uma.

```
NÃO (obteve_vantagem_ilicita E houve_prejuizo_alheio)
```

Negar um `E` não exige negar os dois lados. Basta que um deles falhe: se não houve vantagem ilícita, ou se não houve prejuízo alheio, o conjunto já dá FALSO.

É por isso que sustentar a atipicidade por um único elemento é argumento inteiro, e não meio argumento.

Um segundo artigo, de outra área

Código Civil. Art. 186.

> **Art. 186.** Aquele que, por ação ou omissão voluntária, negligência ou imprudência, violar direito e causar dano a outrem, ainda que exclusivamente moral, comete ato ilícito.

Mesma pergunta que fizemos ao art. 171: o que precisa acontecer junto, e onde basta uma das opções?

O art. 186 vira condição

Nos mesmos grupos, reescrevam o art. 186 como condição, do mesmo jeito: `SE ... ENTÃO ...`, com `E`, `OU` e os parênteses no lugar certo.

Entreguem pelo formulário “art. 186” no Blackboard, com o nome completo de todos os integrantes. Oito minutos.

Aqui uma das vírgulas engana.

O art. 186 vira condição

RESPOSTA

ação **ou** omissão

E

voluntária **ou** por negligência **ou** por imprudência

E

violar direito

E

causar dano a outrem

A vírgula depois de “voluntária” esconde um OU: querer o resultado **ou** ter sido descuidado, qualquer um dos dois basta. E “ainda que exclusivamente moral” não é uma quinta caixa: diz que dano moral já conta como dano.

A mesma forma nos dois artigos

Código Penal. Art. 171, caput.

```
SE   obteve_vantagem_ilicita
 E   houve_prejuizo_alheio
 E   (induziu_em_erro OU manteve_em_erro)
 E   (usou_artificio OU usou_ardil
      OU usou_outro_meio_fraudulento)
ENTÃO
    reclusão de 1 a 5 anos, e multa
```

Código Civil. Art. 186.

```
SE   (agiu OU omitiu_se)
 E   (quis_o_resultado OU foi_negligente
      OU foi_imprudente)
 E   violou_direito
 E   causou_dano
ENTÃO
    cometeu ato ilícito
```

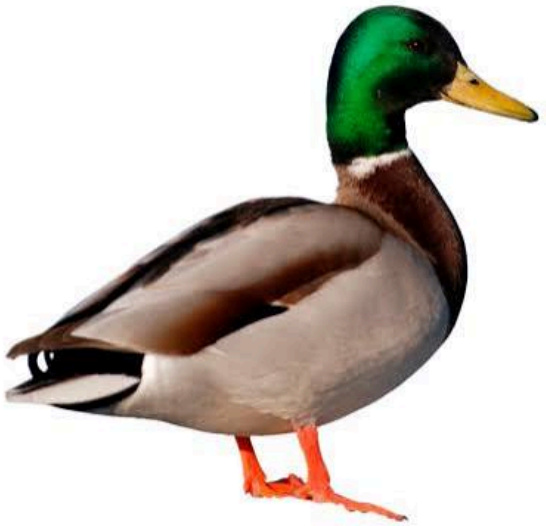
`E` entre os blocos, `OU` dentro deles, parênteses dizendo onde cada `OU` acaba. O texto muda de área, e a forma da condição não.

Quando a lei põe vírgula onde o código põe conectivo, quem formaliza é obrigado a escolher uma leitura, e a escolha fica visível.

Ciência de Dados Aplicada ao Direito I

Aula 8

Loops: `para cada` e `enquanto`



Prof. Bruno da Cunha de Oliveira - 2026/2

Loops

Insper

Isto decide um caso

```
SE data_transito == NULO
 E data_sentenca < 01/09/2024 ENTÃO
    revisar o caso
```

Agora façam isso 200 vezes.

Abrir o processo, ler as duas datas, aplicar a condição, anotar. Duzentas vezes, sem errar nenhuma.

Quanto tempo leva? E se forem dez mil?

Antes do loop: pegar um valor de dentro

Pedir um campo pelo nome da chave

```
decisao = {"idade_autor": 67, "resultado": "PROCEDENTE"}

decisao["idade_autor"]      → 67
decisao["resultado"]        → "PROCEDENTE"
```

O nome do dicionário, e entre colchetes a chave, com as aspas que ela tem. O que volta é o valor guardado nela.

A chave é o “endereço” do valor dentro do dicionário.

Digam como pegar: dicionário

Falem a expressão que pega cada valor.

```
decisao = {
  "numero": "0000000-00.2026.8.26.0100",
  "idade_autor": 67,
  "resultado": "PROCEDENTE",
  "data_transito": NULO
}
```

1. o resultado;
2. a data do trânsito;
3. a idade do réu.

Digam como pegar: dicionário

RESPOSTA

```
decisao["resultado"]       → "PROCEDENTE"
decisao["data_transito"]   → NULO
decisao["idade_reu"]       → não existe
```

A segunda devolve `NULO`, que é um valor guardado. A terceira pede uma chave que o dicionário não tem: isso não devolve vazio, dá erro.

Pedir um item pela posição

```
fundamentos = ["LEI 9.656/98", "SÚMULA 302 STJ", "CDC"]

fundamentos[0]      → "LEI 9.656/98"
fundamentos[1]      → "SÚMULA 302 STJ"
fundamentos[2]      → "CDC"
```

O nome da lista, e entre colchetes a posição, que é um número. A contagem começa em 0: o primeiro item é o 0.

O mesmo colchete: com texto dentro, pede pela chave a um dicionário; com número, pede pela posição a uma lista.

Digam como pegar: lista

Falem a expressão que pega cada item.

```
pecas = ["PETIÇÃO INICIAL", "CONTESTAÇÃO", "RÉPLICA", "SENTENÇA"]
```

1. a petição inicial;
2. a contestação;
3. a sentença.

Digam como pegar: lista

RESPOSTA

```
pecas[0]   → "PETIÇÃO INICIAL"
pecas[1]   → "CONTESTAÇÃO"
pecas[3]   → "SENTENÇA"
```

Quatro itens, e a última está na posição 3, porque a contagem começa em 0. Quem disse `pecas[4]` pediu o que não existe.

Um dentro do outro

LISTA DE DICIONÁRIOS

```
partes = [
  {"nome": "Ana", "papel": "AUTORA"},
  {"nome": "Bruno", "papel": "RÉU"}
]

partes[0]           → {"nome": "Ana", ...}
partes[0]["nome"]   → "Ana"
```

DICIONÁRIO DE LISTAS

```
autos = {
  "numero": "0000000-00.2026.8.26.0100",
  "pecas": ["PETIÇÃO INICIAL", "CONTESTAÇÃO",
"SENTENÇA"]
}

autos["pecas"]      → ["PETIÇÃO INICIAL", ...]
autos["pecas"][2]   → "SENTENÇA"
```

Lê-se da esquerda para a direita: cada colchete pega um valor, e o colchete seguinte pergunta a esse valor.

Peguem três valores

Escrevam a expressão que pega cada um dos três valores, e digam o que acontece na terceira.

```
processo = {
  "numero": "0000000-00.2026.8.26.0100",
  "partes": [
    {"nome": "Ana", "papel": "AUTORA"},
    {"nome": "Bruno", "papel": "RÉU"}
  ],
  "pecas": ["PETIÇÃO INICIAL", "CONTESTAÇÃO", "SENTENÇA"]
}
```

1. o papel do Bruno;
2. a última peça;
3. a quarta peça.

Peguem três valores

RESPOSTA

```
processo["partes"][1]["papel"]   → "RÉU"
processo["pecas"][2]             → "SENTENÇA"
processo["pecas"][3]             → não existe
```

A terceira pede uma posição que a lista não tem. Em pseudocódigo, o robô trava; em Python, o programa para com erro, e a aula 11 mostra qual. Pedir o que não existe não devolve vazio: devolve erro.

Como repetir isto duzentas vezes?

Os duzentos processos estão numa lista chamada `processos`, e cada item dela é um dicionário no formato do codebook. Como vocês formulariam a repetição: aplicar a condição a cada um deles, sem copiar a condição duzentas vezes?

```
SE processo["data_transito"] == NULO
 E processo["data_sentenca"] < 01/09/2024 ENTÃO
    revisar processo
```

Como repetir isto duzentas vezes?

RESPOSTA

Duas formulações possíveis, e as duas são loops:

PARA CADA ITEM DA LISTA

```
PARA CADA processo EM processos:
    SE processo["data_transito"] == NULO
     E processo["data_sentenca"]
         < 01/09/2024 ENTÃO
        revisar processo
```

ENQUANTO HOUVER MAIS

```
ENQUANTO houver processo não visto:
    processo = o próximo não visto
    SE processo["data_transito"] == NULO
     E processo["data_sentenca"]
         < 01/09/2024 ENTÃO
        revisar processo
```

Nas duas, a condição é escrita uma vez e roda duzentas. A da esquerda é a forma para "todos os itens"; a da direita é a forma para "até acabar", e volta na segunda metade da aula.

Para cada

Em Python: `for`

Percorre uma coleção, um item por vez, e roda o bloco indentado para cada item. Termina sozinho quando a coleção acaba.

EXEMPLO

```
PARA CADA parte EM partes:
    ler o nome de parte
```

A lista da aula 6, percorrida

```
partes = [
  {"nome": "Ana", "papel": "AUTORA"},
  {"nome": "Bruno", "papel": "RÉU"}
]

PARA CADA parte EM partes:
    escrever o nome e o papel de parte
```

Vamos ver o robô executar isso, uma volta de cada vez.

A lista da aula 6, percorrida

Animação no navegador: `animacao-loop.html`, cenário `partes`.

A variável de iteração

- O nome é escolha sua. Os três loops abaixo fazem a mesma coisa; o bom nome é o que diz o que cada item é.

```
PARA CADA parte EM partes:      PARA CADA p EM partes:      PARA CADA pessoa EM partes:
```

A variável de iteração

- O nome é escolha sua. Os três loops abaixo fazem a mesma coisa; o bom nome é o que diz o que cada item é.

```
PARA CADA parte EM partes:      PARA CADA p EM partes:      PARA CADA pessoa EM partes:
```

- O que vem depois do `EM` já existe antes do loop: é uma variável com uma coleção dentro. `partes` foi criada lá em cima; `parte` nasce no loop.

A variável de iteração

- O nome é escolha sua. Os três loops abaixo fazem a mesma coisa; o bom nome é o que diz o que cada item é.

```
PARA CADA parte EM partes:      PARA CADA p EM partes:      PARA CADA pessoa EM partes:
```

- O que vem depois do `EM` já existe antes do loop: é uma variável com uma coleção dentro. `partes` foi criada lá em cima; `parte` nasce no loop.
- A variável de iteração vive dentro do bloco indentado. A cada volta o `PARA CADA` põe nela o item seguinte, e você não escreve essa atribuição.

Fora do bloco ela já não é "cada item": o que sobra nela é o último que passou.

Três decisões, uma condição

```
decisoes = [
  {"numero_processo": "0000000-00.2026.8.26.0100", "data_sentenca": 01/09/2026, "data_transito": NULO},
  {"numero_processo": "0000000-01.2023.8.26.0100", "data_sentenca": 15/03/2023, "data_transito": 30/04/2023},
  {"numero_processo": "0000000-02.2024.8.26.0100", "data_sentenca": 10/06/2024, "data_transito": NULO}
]

PARA CADA decisao EM decisoes:
    escrever "checando " + decisao["numero_processo"]
    SE decisao["data_transito"] == NULO
     E decisao["data_sentenca"] < 01/09/2024 ENTÃO
        revisar decisao
```

Três voltas. A linha do `escrever` roda em todas; a do `revisar`, só quando a condição dá VERDADEIRO. Vamos ver na animação.

Três decisões, uma condição

Animação no navegador: `animacao-loop.html`, cenário `decisoes`.

Quantas vezes a linha de dentro roda

EXERCÍCIO

A lista `fundamentos` tem quatro itens. Quantas vezes a linha indentada é executada?

```
fundamentos = ["LEI 9.656/98", "SÚMULA 302 STJ",
               "CDC", "ROL ANS"]

PARA CADA fundamento EM fundamentos:
    escrever fundamento
```

A. uma

B. quatro

C. depende do que estiver guardado em `fundamento`

D. nenhuma: falta a condição de parada

Quantas vezes a linha de dentro roda

RESPOSTA

B. Quatro. Uma por item.

O `PARA CADA` não precisa de condição de parada escrita: ele para quando a coleção acaba. É o que o loop resolve da primeira metade da falha 8 da aula 5, a repetição que não diz quando parar.

Troquei o nome

Reescrevi o loop das decisões trocando o nome da variável. O que muda no resultado?

```
decisoes = [
  {"numero_processo": "0000000-00.2026.8.26.0100", "data_sentenca": 01/09/2026, "data_transito": NULO},
  {"numero_processo": "0000000-01.2023.8.26.0100", "data_sentenca": 15/03/2023, "data_transito": 30/04/2023},
  {"numero_processo": "0000000-02.2024.8.26.0100", "data_sentenca": 10/06/2024, "data_transito": NULO}
]
```

```
PARA CADA x EM decisoes:
    SE x["data_transito"] == NULO
     E x["data_sentenca"] < 01/09/2024 ENTÃO
        revisar x
```

Troquei o nome

RESPOSTA

Nada. O resultado é o mesmo.

O nome da variável de iteração é escolha de quem escreve, e o loop só exige que o mesmo nome seja usado dentro do bloco. O que `x` perde é a leitura: `decisao["data_transito"]` diz o que está sendo lido, `x["data_transito"]` obriga a subir para descobrir.

A linha depois do loop

A última linha está fora do bloco indentado. O que ela escreve?

```
decisoes = [
  {"numero_processo": "0000000-00.2026.8.26.0100", "data_sentenca": 01/09/2026, "data_transito": NULO},
  {"numero_processo": "0000000-01.2023.8.26.0100", "data_sentenca": 15/03/2023, "data_transito": 30/04/2023},
  {"numero_processo": "0000000-02.2024.8.26.0100", "data_sentenca": 10/06/2024, "data_transito": NULO}
]
```

```
PARA CADA decisao EM decisoes:
    SE decisao["data_transito"] == NULO ENTÃO
        revisar decisao

escrever decisao["numero_processo"]
```

A linha depois do loop

RESPOSTA

Só o número da última decisão da lista.

Fora do bloco, o loop já acabou, e `decisao` guarda o último item que passou por ela. Ela já não é “cada decisão”: é uma só. Quem quer fazer algo para cada escreve dentro do bloco.

A linha que escapou do bloco

A ideia era arquivar cada decisão que já transitou em julgado. Alguém apagou a indentação da última linha. O que é arquivado?

```
decisoes = [
  {"numero_processo": "0000000-00.2026.8.26.0100", "data_sentenca": 01/09/2026, "data_transito": NULO},
  {"numero_processo": "0000000-01.2023.8.26.0100", "data_sentenca": 15/03/2023, "data_transito": 30/04/2023},
  {"numero_processo": "0000000-02.2024.8.26.0100", "data_sentenca": 10/06/2024, "data_transito": NULO}
]
```

```
PARA CADA decisao EM decisoes:
    SE decisao["data_transito"] != NULO ENTÃO
        escrever "transitou"
arquivar decisao
```

A linha que escapou do bloco

RESPOSTA

A terceira decisão. E ela não transitou.

“Transitou” é escrito uma vez, na segunda volta. Mas `arquivar decisao` está fora do bloco: roda uma vez, depois do loop, com o que sobrou em `decisao`, que é a terceira. A que transitou, a segunda, fica sem arquivar; a que não transitou vai para o arquivo. O programa não reclama de nada.

A indentação é o que diz o que se repete, e o que ela erra, ninguém avisa.

Parar antes de a lista acabar

Procurar a sentença nas peças. Depois que ela aparece, não há por que olhar o resto.

```
pecas = ["PETIÇÃO INICIAL", "CONTESTAÇÃO", "SENTENÇA", "APELAÇÃO"]

PARA CADA peca EM pecas:
    escrever "olhando " + peca
    SE peca == "SENTENÇA" ENTÃO
        PARAR
```

`PARAR` sai do loop na hora, no meio da volta: “olhando” é escrito três vezes, e a apelação nunca é olhada. Em Python, `break`.

O `para cada` termina quando a lista acaba ou quando alguém manda parar.

Enquanto

Insper

Vai passando até cobrir

Duas instruções da receita da aula 5:

```
vai passando Nutella até cobrir
vai passando Nutella até acabar o pote
```

Dá para escrever cada uma com `PARA CADA`? Para cada o quê?

Vai passando até cobrir

RESPOSTA

ATÉ COBRIR: DÁ

```
partes_do_pao = ["canto 1", "canto 2",
"meio", ...]

PARA CADA parte EM partes_do_pao:
    passar Nutella em parte
```

Dividiu o pão em partes, virou lista. Acaba quando a lista acaba.

ATÉ ACABAR O POTE: NÃO DÁ

Para cada o quê? Não há lista de passadas: ninguém sabe quantas cabem no pote. O número de voltas depende do que acontece em cada uma.

O que falta é a condição de parada escrita junto: repete enquanto ainda houver Nutella.

Enquanto

Em Python: `while`

Repete o bloco indentado enquanto a condição der VERDADEIRO. Testa a condição antes de cada volta, e para na primeira vez em que ela der FALSO.

EXEMPLO

```
ENQUANTO a parte de dentro do pão não estiver coberta
    E ainda houver Nutella no pote:
  passar Nutella numa parte descoberta
```

A variável de controle

```
terminou = FALSO                ← declarada ANTES do loop

ENQUANTO NÃO terminou:          ← lida na condição
    fazer a próxima parte
    SE acabou ENTÃO
        terminou = VERDADEIRO   ← alterada DENTRO do bloco
```

A condição do `ENQUANTO` lê uma variável, e alguma linha dentro do bloco precisa mudar essa variável. Declarada antes, lida na condição, alterada dentro: são as três partes de todo `ENQUANTO`.

Um exemplo de casa: a Nutella

```
coberto = FALSO

ENQUANTO NÃO coberto E ainda houver Nutella no pote:
    passar Nutella numa parte descoberta
    SE não sobrou parte descoberta ENTÃO
        coberto = VERDADEIRO
```

`coberto` é a variável de controle. A segunda metade da condição, o pote, é o que a aula 5 descobriu na prática: a repetição também precisa parar quando o que se usa acaba.

Um exemplo dos autos: a sentença

```
pecas = ["PETIÇÃO INICIAL", "CONTESTAÇÃO", "SENTENÇA", "APELAÇÃO"]
achou_sentenca = FALSO

ENQUANTO NÃO achou_sentenca:
    peca = a próxima peça dos autos
    escrever "olhando " + peca
    SE peca == "SENTENÇA" ENTÃO
        achou_sentenca = VERDADEIRO
```

Ninguém sabe em que peça a sentença está, então não há quantas voltas. O loop abre peça por peça e para na primeira vez em que `achou_sentenca` vira `VERDADEIRO`. Vamos ver o robô executar: animação, cenário 3.

Um exemplo dos autos: a sentença

Animação no navegador: `animacao-loop.html`, cenário `sentenca`.

Para cada ou enquanto

PARA CADA	ENQUANTO
Há uma coleção, e o loop passa por cada item dela.	Há uma condição, e o loop roda enquanto ela for verdadeira.
O número de voltas é o tamanho da coleção.	O número de voltas não se sabe de antemão.
Para sozinho, quando a coleção acaba.	Para quando alguma coisa dentro do bloco faz a condição virar FALSO.

A última linha da tabela é a diferença que importa: no `ENQUANTO`, quem escreve o loop é responsável por ele acabar.

Tirei uma linha

Apaguei a última linha do bloco. O que acontece quando o robô chega à sentença?

```
achou_sentenca = FALSO

ENQUANTO NÃO achou_sentenca:
    abrir a próxima peça dos autos
    SE a peça for a sentença ENTÃO
        registrar a data da sentença
```

Tirei uma linha

RESPOSTA

A data é registrada, a variável não muda, e o loop nunca para.

`achou_sentenca` nasce `FALSO` e nenhuma linha a altera, então `NÃO achou_sentenca` é `VERDADEIRO` para sempre. É o loop infinito: a segunda metade da falha 8 da aula 5, a condição de parada que pode nunca acontecer.

Este processo não tem sentença

O loop está completo, com a linha de volta. Mas este processo ainda não foi sentenciado: as peças acabam e a sentença não aparece. O que acontece, e como vocês consertam?

```
achou_sentenca = FALSO

ENQUANTO NÃO achou_sentenca:
    abrir a próxima peça dos autos
    SE a peça for a sentença ENTÃO
        achou_sentenca = VERDADEIRO
```

Este processo não tem sentença

RESPOSTA

Loop infinito de novo, por outro caminho: a linha de volta existe e nunca roda.

```
ENQUANTO NÃO achou_sentenca E ainda houver peça nos autos:
```

A condição de parada precisa cobrir também o caso em que o que se procura não está lá. É a mesma lição da Nutella que acaba antes de cobrir o pão.

Para cada ou enquanto?

Quando dá para escolher, para cada

Quase tudo que se escreve com `PARA CADA` dá para escrever com `ENQUANTO houver mais`. A recíproca não vale.

SE EXISTE UMA LISTA

`PARA CADA`. Ele acaba sozinho quando a lista acaba, e não há variável de controle para esquecer de mudar.

SE O NÚMERO DE VOLTAS DEPENDE DO QUE ACONTECE

`ENQUANTO`. Não há lista para percorrer: há uma condição que muda a cada volta, e o loop para quando ela cai.

No quiz a pergunta é: qual dos dois é o natural para o caso? Se a lista existe antes de começar, é o `para cada`.

As testemunhas arroladas

Ouvir cada uma das testemunhas arroladas pela defesa.

A. PARA CADA

B. ENQUANTO

As testemunhas arroladas

RESPOSTA

A. PARA CADA.

O rol de testemunhas é uma coleção que existe antes de começar, e o número de voltas é o tamanho dele. Daria com `ENQUANTO houver testemunha não ouvida`, mas aí alguém precisa cuidar da parada; o `PARA CADA` cuida sozinho.

O documento que falta

Intimar a parte para juntar o documento, de novo, até ela juntar.

A. PARA CADA

B. ENQUANTO

O documento que falta

RESPOSTA

B. ENQUANTO.

Não há coleção de intimações: há uma condição, “o documento ainda não veio”, e ninguém sabe quantas vezes vai repetir. E vale a pergunta da Nutella: o que faz esse loop parar se a parte nunca juntar?

As cláusulas do contrato

Conferir, em cada cláusula do contrato, se há multa acima do teto legal.

A. PARA CADA

B. ENQUANTO

As cláusulas do contrato

RESPOSTA

A. PARA CADA.

As cláusulas são uma coleção, e o `SE` da multa fica dentro do bloco, como a condição de caducidade ficou dentro do loop das decisões.

Os endereços do réu

Tentar a citação em todos os endereços conhecidos do réu, até ele ser encontrado.

A. PARA CADA

B. ENQUANTO

Os endereços do réu

RESPOSTA

A. PARA CADA, com PARAR.

Os endereços conhecidos são uma lista que existe antes de começar. O “até” não muda isso: é o `PARAR` quando o réu for encontrado, como a sentença nas peças. Se ele não for encontrado em nenhum, o loop acaba sozinho no último endereço.

A distribuição do dia

Verificar, em cada processo distribuído hoje, se a petição inicial veio com procuração.

A. PARA CADA

B. ENQUANTO

A distribuição do dia

RESPOSTA

A. PARA CADA.

A lista do dia está fechada quando o robô começa. Se a instrução fosse “verificar os processos conforme forem chegando, até o plantão acabar”, aí seria `ENQUANTO`.